



# 3DVerifier: efficient robustness verification for 3D point cloud models

Ronghui Mu<sup>1</sup> · Wenjie Ruan<sup>2</sup> · Leandro S. Marcolino<sup>1</sup> · Qiang Ni<sup>1</sup>

Received: 30 May 2022 / Revised: 9 August 2022 / Accepted: 12 September 2022 /  
Published online: 2 November 2022  
© The Author(s) 2022

## Abstract

3D point cloud models are widely applied in safety-critical scenes, which delivers an urgent need to obtain more solid proofs to verify the robustness of models. Existing verification method for point cloud model is time-expensive and computationally unattainable on large networks. Additionally, they cannot handle the complete PointNet model with joint alignment network that contains multiplication layers, which effectively boosts the performance of 3D models. This motivates us to design a more efficient and general framework to verify various architectures of point cloud models. The key challenges in verifying the large-scale complete PointNet models are addressed as dealing with the cross-non-linearity operations in the multiplication layers and the high computational complexity of high-dimensional point cloud inputs and added layers. Thus, we propose an efficient verification framework, 3DVerifier, to tackle both challenges by adopting a linear relaxation function to bound the multiplication layer and combining forward and backward propagation to compute the certified bounds of the outputs of the point cloud models. Our comprehensive experiments demonstrate that 3DVerifier outperforms existing verification algorithms for 3D models in terms of both efficiency and accuracy. Notably, our approach achieves an orders-of-magnitude improvement in verification efficiency for the large network, and the obtained certified bounds are also significantly tighter than the state-of-the-art verifiers. We release our tool 3DVerifier via <https://github.com/TrustAI/3DVerifier> for use by the community.

**Keywords** Robustness verification · 3D point cloud models · Adversarial attacks · Deep neural networks · PointNet

## 1 Introduction

Recent years have witnessed increasing interest in 3D object detection, and Deep Neural Networks (DNNs) have also demonstrated their remarkable performance in this area (Qi et al., 2017a, b). For 3D object detectors, the point clouds are utilized to represent 3D

---

Editors: Yu-Feng Li and Prateek Jain.

---

✉ Wenjie Ruan  
w.ruan@exeter.ac.uk

Extended author information available on the last page of the article

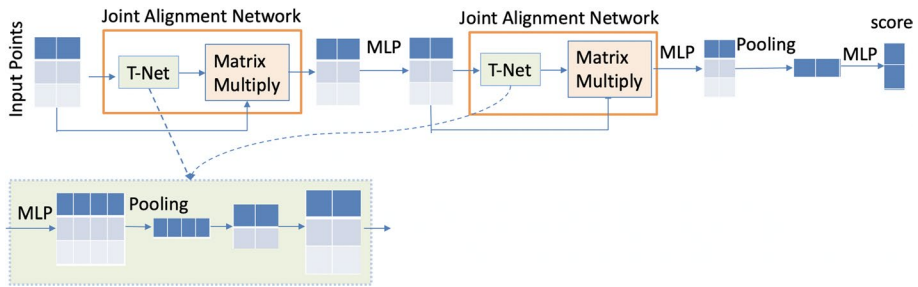
objects, which are usually the raw data gained from LIDARs and depth cameras. Such 3D deep learning models have been widely employed in multiple safety-critical applications such as motion planning (Varley et al., 2017), virtual reality (Stets et al., 2017), and autonomous driving (Chen et al., 2017; Liang et al., 2018). However, extensive research has shown that DNNs are vulnerable to adversarial attacks, appearing as adding a small amount of non-random, ideally human-invisible, perturbations on the input will cause DNNs to make abominable predictions (Szegedy et al., 2013; Carlini et al., 2017; Jin et al., 2020). Therefore, there is an urgent need to address such safety concerns on DNNs caused by adversarial examples, especially on safety-critical 3D object detection scenarios.

Recently, most works on analyzing the robustness of 3D models mainly focus on adversarial attacks with an aim to reveal the model's vulnerabilities under different types of perturbations, such as adding or removing points, and shifting positions of points (Liu et al., 2019; Yang et al., 2019). In the meanwhile, adversarial defenses are also proposed to detect or prevent these adversarial attacks (Yang et al., 2019; Zhou et al., 2019). However, as Tramer et al. (2020) and Athalye et al. (2018) indicated, *even though these defenses are effective for some attacks, they still can be broken by other stronger attacks*. Thereby, we need a more solid solution, ideally with *provable guarantees*, to verify whether the model is robust to *any* adversarial attacks within an allowed perturbation budget.<sup>1</sup> This technique is also generally regarded as verification on (local) adversarial robustness<sup>2</sup> in the community. So far, various solutions have been proposed to tackle robustness verification, but they mostly focus on the image domain (Boopathy et al., 2019; Singh et al., 2019; Tjeng et al., 2017; Jin et al., 2022). By contrast, verifying the adversarial robustness of 3D point cloud models is barely explored by the community. As far as we know, 3DCertify, proposed by Lorenz et al. (2021) is the first, and also the *only* work to verify the robustness of 3D models. Although 3DCertify is very inspiring, it has yet completely resolved some critical challenges on robustness verification for 3D models, according to our empirical study.

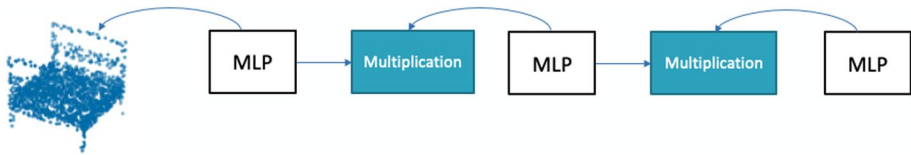
Below we address three main challenges: Firstly, so far 3DCertify is the first and only verification tool for 3D models, the empirical experiments indicate that it is time-consuming and thus not computationally attainable on large neural networks. As 3DCertify is built upon DeepPoly (Singh et al., 2019), when directly applying the relaxation algorithm that is specifically designed for images on the high-dimensional point clouds, it will result in out-of-memory issues and cause the termination of the verification. The second challenge is that there is no tool to verify the JANet in the PointNet. 3DCertify can only verify a simplified PointNet model without Joint Alignment Network (JANet) consisting of matrix multiplication operations. Figure 1 illustrates the abstract architecture of a complete PointNet (Qi et al., 2017a), one of the most widely used models on 3D object detection. As we can see, since the learnt representations are expected to be invariant to spatial transformations, JANet is the key enabler in PointNet to achieve this geometric invariant functionality by adopting the T-Net and matrix multiplications. Recent research also demonstrates that JANet is an essential for boosting the performance of PointNet (Qi et al., 2017a) and thus widely applied in some safety-critical tasks (Paigwar et al., 2019; Aoki et al., 2019; Chen et al., 2021). Thirdly, 3DCertify can only work on  $l_\infty$ -norm metric, however, arguably, some researchers in the community regard other  $l_p$ -norm metrics such as  $l_1$ ,  $l_2$ -norm

<sup>1</sup> In the community, we normally use a small predefined  $l_p$ -norm ball to quantify such perturbations, namely, within this small perturbing space the decision should remain the same from a perspective of a human observer.

<sup>2</sup> For convenience, we use *robustness verification* or *verification* for short in this paper.



**Fig. 1** The abstract framework of the PonitNet with joint alignment network (JANet), where MLP stands for multi-layer perceptron



**Fig. 2** Illustration of the combining forward and backward propagation process. The architecture in the MLP block contains convolution with ReLU activation function, batch normalization and pooling. Inside the MLP block, the bounds are computed by backward propagation

metrics are equally (if not more) important in the study of adversarial robustness (Boopathy et al., 2019; Weng et al., 2018). Thus, a robustness verification tool that can work on a wide range of  $l_p$ -norm metrics is also worthy of comprehensive exploration.

Thus, motivated by the aforementioned challenges yet to be resolved, this paper aims to design an *efficient* and *scalable* robustness verification tool that can handle a wide range of 3D models, including those with JANet structures under multiple  $l_p$ -norm metrics including  $l_\infty$ ,  $l_1$ , and  $l_2$ -norm. We achieve the verification efficiently by adapting the efficient layer-by-layer certification framework used in (Weng et al., 2018; Boopathy et al., 2019). Considering that these verifiers are designed for images and cannot be applied in larger-scale 3D point cloud models, we introduce a novel relaxation function of global max pooling to make it applicable and efficient on PointNet. Moreover, the multiplication layers in the JANet structure involves two variables under perturbations, which brings the cross-non-linearity. Due to the high dimensionality in 3D models, such cross-non-linearity results in significant computational overhead for computing a tight bound. To solve the cross-non-linearity, Shi et al. (2020) proposed to use a closed-form linear function to bound the multiplication layer in the attention layers of Transformer. As the JANet includes multiplication between two variables, where one variable is the output of the previous layer, which makes it different from that in Transformer, we propose to use the closed-form linear functions to bound the multiplication layer and combine forward and backward propagation, which can also benefit the computation cost by calculating the bound in only  $O(1)$  complexity. The process is demonstrated in Fig. 2.

In summary, the main contributions of this paper are listed below.

- Our method can achieve an efficient verification. We design a relaxation algorithm to resolve the cross-non-linearity challenge by combining the forward and backwards propagation, enabling an efficient yet tight verification on matrix multiplications.

- We design an efficient and scalable verification tool, 3DVerifier, with provable guarantees. It is a general framework that can verify the robustness for a wide range of 3D model architectures, especially it can work on complete and large-scale 3D models under  $l_\infty$ ,  $l_1$ , and  $l_2$ -norm perturbations.
- 3DVerifier, as far as we know, is one of the very few works on 3D model verification, which is more advanced than the existing work, 3DCertify, in terms of efficiency, scalability and tightness of the certified bounds.

## 2 Related work

*3D Deep Learning Models:* As the raw data of point clouds comes directly from real-world sensors, such as LIDARs and depth cameras, point clouds are widely used to represent 3D objects for the deep learning classification task. To deal with the unordered point clouds, Qi et al. (2017a) proposed the PointNet model, which utilised the global max-pooling layer to assemble features of points and the joint alignment network (JANet). Then, Qi et al. (2017b) extended the PointNet by using the spacial neighbor graphs.

*Adversarial Attacks on 3D Point Clouds Classification:* Szegedy et al. (2013) first proposed the concepts of adversarial attacks and indicated that the neural networks are vulnerable to the well-crafted imperceptible perturbation. Xiang et al. (2019) claimed that they are the first to perform extensive adversarial attacks on 3D point cloud models by perturbing the positions of points or generating new points. Recent works extended the adversarial attacks for images to 3D point clouds by perturbing points and generating new points (Goodfellow et al., 2014; Yang et al., 2019; Lee et al., 2020). Additionally, Cao et al. (2019) proposed adversarial attacks on LiDAR systems. Wicker and Kwiatkowska (2019) used occlusion attack and Zhao et al. (2020) proposed isometric transformations attack. Towards these adversarial attacks, corresponding defense techniques are developed (Yang et al., 2019; Zhou et al., 2019), which are more effective than adversarial training such as (Liu et al., 2019; Zhang et al., 2019). However, Sun et al. (2020) examined existing defense works and pointed out that those defenses are still vulnerable to more powerful attacks. Thus, Lorenz et al. (2021) proposed the first verification algorithm for 3D point cloud models with provable robustness bounds.

*Verification on Neural Networks:* The robustness verification aims to find a guaranteed region that any input in this region leads to the same predicted label. For image classification, the region is bounded by a  $l_p$ -norm ball with a radius of  $\epsilon$ , aiming to maximize the  $\epsilon$ . For point clouds models, we reformulate the goal to find the maximum  $\epsilon$  that any distortion applied on the position of the point within the region cannot alter the predicted label. Numerous works have attempted to find the exact value in the image domain  $\epsilon$  (Katz et al., 2017; Tjeng et al., 2017; Bunel et al., 2018). Yet these approaches are designed for small networks. Some works focus on computing a certified lower bound for the  $\epsilon$ . To handle the non-linearity operations in the neural network, the convex relaxation is proposed to approximate the bounds for the ReLU layer (Salman et al., 2019). Wong and Kolter (2018) and Dvijotham et al. (2018) introduced a dual approach to form the relaxation. Several studies computed the bounds via linear-by-linear approximation (Wang et al., 2018; Weng et al., 2018; Zhang et al., 2018) or abstract domain interpretation (Gehr et al., 2018; Singh et al., 2019).

While for the 3D point cloud models, only Lorenz et al. (2021) proposed the 3DCertify. However, their method is time expensive and can not handle the JANet in the full PointNet

models. Thus, in this paper, we build a more efficient and general verification framework to obtain the certified bounds.

### 3 Methodology: 3DVerifier

#### 3.1 Overview

The clean input point cloud  $\mathbf{P}_0$  with  $n$  points can be defined as  $\mathbf{P}_0 = \{\mathbf{p}_0^{(i)} \mid \mathbf{p}_0^{(i)} \in \mathbb{R}^3, i = 1, \dots, n\}$ , where each point  $\mathbf{p}_0^{(i)}$  is represented in a 3D space coordinate  $(x, y, z)$ . We choose the points that are correctly recognized by the classifier  $C$  as inputs to verify the robustness of  $C$ .

Throughout this paper, we will perturb the points by shifting their positions in the 3D space within a distance bounded by the  $l_p$ -norm ball. Given the perturbed input  $\mathbf{P} = \{\mathbf{p}^{(i)} \mid \mathbf{p}^{(i)} \in \mathbb{R}^3, i = 1, \dots, n\}$  that is in the  $l_p$ -norm ball  $\mathbb{S}_p(\mathbf{p}_0^{(i)}, \epsilon) := \{\mathbf{p}^{(i)} \mid \|\mathbf{p}^{(i)} - \mathbf{p}_0^{(i)}\|_p \leq \epsilon, i = 1, \dots, n\}$ , we aim to verify whether the predicted label of the model is stable within the region  $\mathbb{S}_p$ . This can be solved by finding the **minimum adversarial perturbation**  $\epsilon_{\min}$  via binary search, such that  $\exists \mathbf{P} \in \mathbb{S}_p(\mathbf{P}_0, \epsilon_{\min})$ ,  $\text{argmax } C(\mathbf{P}) \neq c$ , where  $c = \text{argmax } C(\mathbf{P}_0)$ . Such  $\epsilon_{\min}$  is also referred as the *untargeted* robustness. As for the *targeted* robustness, it can be interpreted as that the prediction output score for the true class is always greater than that for the target class.

Assuming that the target class is  $t$ , the objective function is:

$$\min \{y_c(\mathbf{P}) - y_t(\mathbf{P})\} := \sigma_\epsilon, \quad s.t. \quad \|\mathbf{p}^k - \mathbf{p}_0^k\|_p \leq \epsilon, (k = 1, 2, \dots, n), \quad (1)$$

where  $y_c$  represents the logit output for class  $c$  and  $y_t$  is for the target class  $t$ .  $\mathbf{P}$  is the set of points that is centred around the original points set  $\mathbf{P}_0$  within the  $l_p$ -norm ball with radius  $\epsilon$ . Thus, if  $\sigma_\epsilon > 0$ , the logit output of the true class  $c$  is always greater than the target class, which means that the predicted label cannot be  $t$ . Due to the fact that finding the exact output of  $\sigma_\epsilon$  is an NP-hard problem (Katz et al., 2017), the objective function of our work can be alternatively altered as computing the lower bound of  $\sigma_\epsilon$ . By applying binary search to update the perturbation  $\epsilon$ , we can find the minimum adversarial perturbation. Equivalently, the *maximum*  $\epsilon_{\text{cert}}$  that does not alter the predicted label can be attained. Thus, in this paper, we aim to compute the **certified lower bound** of  $\sigma_{\epsilon_{\text{cert}}}$ .

#### 3.2 Generic framework

To obtain the lower bound of  $\sigma_\epsilon$  for the  $l_p$ -norm ball bounded model, we propagate the bound of each neuron layer by layer. As mentioned previously, most structures employed by the 3D point cloud models are similar to the traditional image classifiers, such as the convolution layer, batch normalization layer, and pooling layer. The most distinctive structure of the point clouds classifier, like the PointNet (Qi et al., 2017a), is the JANet structure. Thus, to compute the logit outputs of the neural network, our verification algorithm adopt three types of formulas to handle different operations: (1) linear operations (e.g. convolution, batch normalization and average pooling), (2) non-linear operation (e.g. ReLU activation function and max pooling), (3) multiplication operation.

Let  $\Phi^l(\mathbf{P})$  be the output of neurons in the layer  $l$  with point clouds input  $\mathbf{P}$ . The input layer can be represented as  $\Phi^0(\mathbf{P}) = \mathbf{P}$ . Suppose that the total number of layers in the classifier is  $m$ ,  $\Phi^m(\mathbf{P})$  is defined as the output of the neural network. In order to verify the classifier, we aim to derive a linear function to obtain the global upper bound  $u$  and lower bound  $l$  for each layer output  $\Phi^l(\mathbf{P})$  for  $\mathbf{P} \in \mathbb{S}_p(\mathbf{P}_0, \epsilon)$ .

The bounds are derived layer by layer from the first layer to the final layer. We have full access to all parameters of the classifier, such as weights  $\mathbf{W}$  and bias  $\mathbf{b}$ . To calculate the output for each neuron, we show below the linear functions to obtain the bounds of the  $l$ -th layer for the neuron in position  $(x, y)$  based on the previous  $l'$ -th layer:

$$\sum_{ij} \mathbf{A}_{(x,y,i,j)}^{(l,l'),L} \Phi_{(x+i,j)}^{l'}(\mathbf{P}) + \mathbf{B}_{(x,y)}^{(l,l'),L} \leq \Phi_{(x,y)}^l(\mathbf{P}) \leq \sum_{ij} \mathbf{A}_{(x,y,i,j)}^{(l,l'),U} \Phi_{(x+i,j)}^{l'}(\mathbf{P}) + \mathbf{B}_{(x,y)}^{(l,l'),U}, \quad (2)$$

where  $\mathbf{A}^L, \mathbf{B}^L, \mathbf{A}^U, \mathbf{B}^U$  are weight and bias matrix parameters of the linear function for lower and upper bound calculations respectively.  $\mathbf{A}$  and  $\mathbf{B}$  are initially assigned as identity matrix ( $\mathbf{I}$ ) and zeros matrix ( $\mathbf{0}$ ), respectively, to keep the same output of  $\Phi_{(x,y)}^l$  when  $l = l'$ . To calculate the bounds of current layer, we take backward propagation to previous layers. The  $\Phi_{(x+i,j)}^{l'}$  is substituted by the linear function of the previous layer recursively until it reaches the first layer ( $l' = 0$ ). After that, the output of each layer can be formed by a linear function of the first layer ( $\Phi^0(\mathbf{P}) = \mathbf{P}$ ), as:

$$\mathbf{A}^{(l,0),L} * \mathbf{P} + \mathbf{B}^{(l,0),L} \leq \Phi^l(\mathbf{P}) \leq \mathbf{A}^{(l,0),U} * \mathbf{P} + \mathbf{B}^{(l,0),U}. \quad (3)$$

Since the perturbation added to the point clouds input is bounded by the  $l_p$ -norm ball,  $\mathbf{p} \in \mathbb{S}_p(\mathbf{p}_0, \epsilon)$ , to compute the global bounds we need to minimize the lower bound and maximize the upper bound in Eq. 3 over the input region. Thereby, the linear function to compute the global bounds for the  $l$ -th layer can be represented as:

$$\Phi_{x,y}^{l,U/L} = \pm \epsilon \left\| \mathbf{A}_{(x,y, :, :)}^{(l,0),U/L} \right\|_q + \sum_{ij} \mathbf{A}_{(x,y,i,j)}^{(l,0),U/L} \mathbf{p}_0^{(x+i,j)} + \mathbf{B}_{(x,y)}^{(l,0),U/L}, \quad (4)$$

where  $\|\mathbf{A}\|_q$  is the  $l_q$ -norm of  $\mathbf{A}$  and  $1/p + 1/q = 1$  with  $p, q > 1$ , “U/L” denotes that the equations are formulated for the upper bounds and lower bounds, respectively. This generic framework resembles to CROWN (Zhang et al., 2018), which is widely utilised in the verification works to verify feed-forward neural networks (e.g. CNN-Cert Boopathy et al., 2019, Transformer Verification Shi et al., 2020). Unlike existing frameworks based on CROWN, we further extend the algorithm to verify point clouds classifiers.

### 3.3 Functions for linear and non-linear operation

As the linear and nonlinear functions are basic operations in the neural network, we first adapt the framework given in (Boopathy et al., 2019) to 3D point cloud models. In Sect. 3.4, we will present our novel technique for JANet.

*Functions for linear operation.* Suppose the output of the  $l'$ -th layer,  $\Phi^{l'}(\mathbf{P})$ , can be computed by the output of the  $(l'-1)$ -th layer,  $\Phi^{l'-1}(\mathbf{P})$ , by the linear function  $\Phi^{l'}(\mathbf{P}) = \mathbf{W}^{l'} * \Phi^{l'-1}(\mathbf{P}) + \mathbf{b}^{l'}$ , where  $\mathbf{W}^{l'}$  and  $\mathbf{b}^{l'}$  are parameters of the function in the layer  $l'$ . Thus the Eq. 2 can be propagated from the layer  $l$  to the layer  $l' - 1$  by substituting  $\Phi^{l'}(\mathbf{P})$ .

$$\begin{aligned}
\Phi_{(x,y)}^{l,U/L}(\mathbf{P}) &= \sum_{i,j} \mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L} \Phi_{(x+i,j)}^{l'}(\mathbf{P}) + \mathbf{B}_{(x,y)}^{(l,l'),U/L} \\
&= \sum_{i,j} \mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L} \left( \sum_k \mathbf{W}_{(k,j)}^{l'} \Phi_{(x+i,k)}^{l'-1}(\mathbf{P}) + \mathbf{b}_{x+i,j}^{l'} \right) + \mathbf{B}_{(x,y)}^{(l,l'),U/L} \\
&= \sum_{i,k} \mathbf{A}_{(x,y,i,k)}^{(l,l'-1),U/L} \Phi_{(x+i,k)}^{l'-1}(\mathbf{P}) + \mathbf{B}_{(x,y)}^{(l,l'-1),U/L}
\end{aligned}$$

*Functions for basic non-linear operation.* For the  $l'$ -th layer with non-linear operations, we apply two linear functions to bound  $\Phi^{(l')}(\mathbf{P})$ :

$$\alpha^{l',L} \Phi^{l'-1}(\mathbf{P}) + \beta^{l',L} \leq \Phi^{(l')}(\mathbf{P}) \leq \alpha^{l',U} \Phi^{l'-1}(\mathbf{P}) + \beta^{l',U}.$$

Given the bounds of  $\Phi^{(l'-1)}(\mathbf{P})$ , the corresponding parameter  $\alpha^L, \alpha^U, \beta^L, \beta^U$  can be chosen appropriately. The functions to obtain the parameters are presented in “Appendix A”.

After computing the corresponding  $\alpha^L, \alpha^U, \beta^L, \beta^U$ , we back propagate the Eq. 2 to the  $(l'-1)$ -th layer:

$$\begin{aligned}
\Phi_{(x,y)}^{l,U/L}(\mathbf{P}) &= \sum_{i,j} \mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L} \Phi_{(x+i,j)}^{l'}(\mathbf{P}) + \mathbf{B}_{(x,y)}^{(l,l'),U/L} \\
&= \sum_{i,j} \mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L,+} * (\alpha_{(x+i,j)}^{l',U/L} \Phi_{(x+i,j)}^{l'-1}(\mathbf{P}) + \beta_{(x+i,j)}^{l',U/L}) \\
&\quad + \sum_{i,j} \mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L,-} * (\alpha_{(x+i,j)}^{l',L/U} \Phi_{(x+i,j)}^{l'-1}(\mathbf{P}) + \beta_{(x+i,j)}^{l',L/U}) \\
&= \sum_{i,j} \mathbf{A}_{(x,y,i,j)}^{(l,l'-1),U/L} \Phi_{(x+i,j)}^{l'}(\mathbf{P}) + \mathbf{B}_{(x,y)}^{(l,l'-1),U/L},
\end{aligned}$$

where if  $\mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L}$  is a positive element of  $\mathbf{A}_{(:, :, :, j)}^{(l,l')}$ , then  $\mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L,+} = \mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L}$  and  $\mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L,-} = 0$ ; otherwise,  $\mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L,-} = \mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L}$  and  $\mathbf{A}_{(x,y,i,j)}^{(l,l'),U/L,+} = 0$ .

### 3.4 Functions for multiplication layer

The most critical structure in the PointNet model is the JAnet, which contains the multiplication layers. For the multiplication, assume that it takes the output of previous layer ( $\Phi^{l'-1}(\mathbf{P})$ ) and  $(l'-r)$ -th layer ( $\Phi^{l'-r}(\mathbf{P})$ ,  $r \in [1, l']$ ) as inputs, the output of  $\Phi^l(\mathbf{P})$  can be calculated by:

$\Phi_{(x,y)}^{l'}(\mathbf{P}) = \sum_{k=1}^{d_k} \Phi_{(x,k)}^{l'-r}(\mathbf{P}) * \Phi_{(k,y)}^{l'-1}(\mathbf{P})$ , where  $d_k$  is the dimension of  $\Phi_{(x,:)}^{l'-r}(\mathbf{P})$  and  $\Phi_{(:,y)}^{l'-1}(\mathbf{P})$ . As the Transformer contains multiplication operation in self-attention layers, inspired by the Transformer verifier (Shi et al., 2020), we proposed our algorithm for point clouds models.

In the JAnet, before the multiplication layer, there is one reshape layer and one pooling layer. To simplify the computation, we choose the  $\Phi^{l'-1}$  as the output of pooling layer, by using  $h = d_k * (k-1) + y$  to represent the transformation in the reshape layer. The equation to compute  $\Phi^{l'-1}(\mathbf{P})$  can be rewritten as:  $\Phi_{(x,y)}^{l'}(\mathbf{P}) = \sum_{k=1}^{d_k} \Phi_{(x,k)}^{l'-r}(\mathbf{P}) * \Phi_{(0,h)}^{l'-1}(\mathbf{P})$ , where  $h = d_k * (k-1) + y$ .

To obtain the bounds of the multiplication layer, we utilize two linear functions of the input  $\mathbf{P}$  to bound  $\Phi^l(\mathbf{P})$ :

$$\Lambda_{(x,y,i,:)}^{(l',0),L} \mathbf{p}^{(x+i)} + \Theta_{(x,y)}^{(l',0),L} \leq \Phi_{(x,y)}^{l'}(\mathbf{P}) \leq \Lambda_{(x,y,i,:)}^{(l',0),U} \mathbf{p}^{(x+i)} + \Theta_{(x,y)}^{(l',0),U} \quad (5)$$

where  $\lambda$  and  $\Theta$  are new parameters of linear functions to bound the multiplication.

**Theorem 1** Let  $l_r$  and  $u_r$  be the lower and upper bounds of the  $(l'-r)$ -th layer output ( $\Phi^{l'-r}(\mathbf{P})$ ,  $r \in [1, l']$ );  $l_1$  and  $u_1$  be the lower and upper bounds of the  $(l'-1)$ -th layer output ( $\Phi^{l'-1}(\mathbf{P})$ ). Suppose  $a^L = l_1$ ,  $a^U = u_1$ ,  $b^L = b^U = l_r$ ,  $c^L = -l_r * l_1$ , and  $c^U = -l_r * u_1$ . Then, for any point clouds input  $\mathbf{P} \in \mathbb{S}_p(\mathbf{P}_0, \epsilon)$ :

$$a^L * \Phi_{(x,k)}^{l'-r}(\mathbf{P}) + b^L * \Phi^{l'-1}(\mathbf{P}) + c^L \leq \Phi_{(x,k)}^{l'-r}(\mathbf{P}) * \Phi_{(k,y)}^{l'-1}(\mathbf{P}) \leq a^U * \Phi_{(x,k)}^{l'-r}(\mathbf{P}) + b^U * \Phi_{(0,h)}^{l'-1}(\mathbf{P}) + c^U,$$

**Proof** see detailed proof in “Appendix B”.  $\square$

The bounds and corresponding bounds matrix of  $\Phi^{l'-r}(\mathbf{P})$  and  $\Phi^{l'-1}(\mathbf{P})$  can be calculated via back propagation. Given Theorem 1, the functions can be formed to compute the  $\Lambda_{(l',0),U/L}$  and  $\Theta_{(l',0),U/L}$  for  $\Phi_{(x,y)}^{l'}(\mathbf{P})$  in Eq. 5 as:

$$\begin{aligned} \Lambda_{(x,y,i,:)}^{(l',0),U/L} &= \sum_k (a_{(0,h)}^{U/L,+} \Lambda_{(x,k,i,:)}^{(l'-r,0),U/L} + a_{(0,h)}^{U/L,-} \Lambda_{(x,k,i,:)}^{(l'-r,0),L/U}) \\ &\quad + \sum_k (b_{(x,k)}^{U/L,+} \Lambda_{(0,h,i,:)}^{(l'-1,0),U/L} + b_{(x,k)}^{U/L,-} \Lambda_{(0,h,i,:)}^{(l'-1,0),L/U}), \\ \Theta_{(x,y)}^{(l',0),U/L} &= \sum_k (a_{(0,h)}^{U/L,+} \mathbf{B}_{(x,k)}^{(l'-r,0),U/L} + a_{(0,h)}^{U/L,-} \mathbf{B}_{(x,k)}^{(l'-r,0),L/U}) \\ &\quad + \sum_k (b_{(x,k)}^{U/L,+} \mathbf{B}_{(k,y)}^{(l'-1,0),U/L} + b_{(x,k)}^{U/L,-} \mathbf{B}_{(0,h)}^{(l'-1,0),U/L}) + c_{(x,y)}^{U/L}. \end{aligned}$$

Thereby, it is a forward propagation process by employing the computed bounds metrics of  $\Phi^{l'-r}(\mathbf{P})$  and  $\Phi^{l'-1}(\mathbf{P})$  to obtain the bounds of  $\Phi^{l'}(\mathbf{P})$ . When it comes to the later layer, the  $l$ -th layer, we use the backward process to propagate the bounds to the multiplication layer, which can be referred as the  $l'$ -th layer. Next, at the multiplication layer, we propagate the bounds to the input layer directly by skipping previous layers. The bounds propagating to the multiplication layer ( $l'$ -th layer) can be represented by the Eq. 2. Therefore,  $\Phi^{l'}(\mathbf{P})$  can be substituted by the linear functions in Eq. 5 to obtain  $\Lambda_{(l,0),U/L}$  and  $\mathbf{B}_{(l,0),U/L}$ :

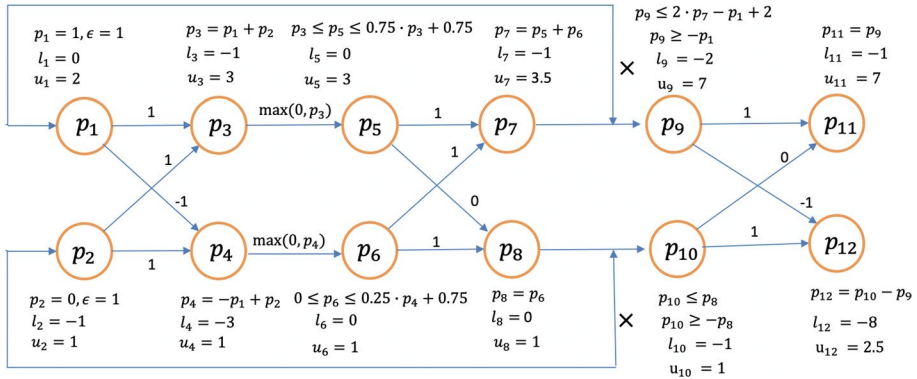
$$\begin{aligned} \Lambda_{(x,y,i,:)}^{(l,0),U/L} &= \sum \left( \Lambda_{(x,y,i,:)}^{(l,l'),U/L,+} \Lambda_{(x+i,y,,:)}^{(l',0),U/L} + \Lambda_{(x,y,i,:)}^{(l,l'),U/L,-} \Lambda_{(x+i,y,,:)}^{(l',0),L/U} \right), \\ \mathbf{B}_{(x,y)}^{(l,0),U/L} &= \mathbf{B}_{(x,y)}^{(l,l'),U/L} + \sum (\Lambda_{(x,y,i,:)}^{(l,l'),U/L,+} \Theta_{(x+i,y)}^{(l',0),U/L} + \Lambda_{(x,y,i,:)}^{(l,l'),U/L,-} \Theta_{(x+i,y)}^{(l',0),L/U}). \end{aligned}$$

Lastly, the global bounds of the  $l$ -th layer can be computed using the linear functions in Eq. 4.

### 3.5 A running numerical example

To demonstrate the verification algorithm, we present a simple example network in Fig. 3 with two input points,  $p_1$  and  $p_2$ . Suppose the input points are bounded by a  $l_\infty$ -norm ball with radius  $\epsilon$ , our goal is to compute the lower and upper bounds of the outputs ( $p_{11}, p_{12}$ ) based on the input intervals. As Fig. 3 shows, the neural network contains 12 nodes,





**Fig. 3** A running example for a simple neural network with multiplication. The inputs ( $p_1, p_2$ ) are bounded by a  $l_\infty$ -norm ball with radius  $\epsilon$

and each node is assigned a weight variable. There are three types of operations in the example network: linear operation, non-linear operation (ReLU activation function) and multiplication.

Given the input points  $\mathbf{P} = [p_1 = 1, p_2 = 0]$ , to obtain the bounds for  $p_3$  and  $p_4$ , according to the Eq. 2,  $\mathbf{A}^{(1,0)}$  can be assigned as  $\mathbf{A}_{(0,0,::)}^{(1,0)} = [1 \ 1]$  and  $\mathbf{A}_{(0,1,::)}^{(1,0)} = [-1 \ 1]$  to compute the output for the 1-st layer  $[p_3, p_4]$ :

$$p_3 = p_1 \cdot \mathbf{A}_{(0,0,0,0)}^{(1,0)} + p_2 \cdot \mathbf{A}_{(0,0,0,1)}^{(1,0)} = 1, p_4 = p_1 \cdot \mathbf{A}_{(0,1,0,0)}^{(1,0)} + p_2 \cdot \mathbf{A}_{(0,1,0,1)}^{(1,0)} = -1. \quad (6)$$

Based on Eq. 4, we can obtain the lower bound and upper bound for the perturbed input ( $p = \infty$ ):  $l_3 = 1 - 2\epsilon = -1$ ,  $u_3 = 1 + 2\epsilon = 3$ ,  $l_4 = -1 - 2\epsilon = -3$ , and  $u_4 = -1 + 2\epsilon = 1$ .

For the non-linear ReLU activation layer, according to the relaxation rule demonstrated in “Appendix A”, in our example, for  $p_5$ , we obtain  $\alpha_L = 1$ ,  $\beta_L = 0$ ,  $\alpha_U = 0.75$ , and  $\beta_U = 0.75$ ; for  $p_6$ , we choose  $\alpha_L = \beta_L = 0$ ,  $\alpha_U = 0.25$ , and  $\beta_U = 0.75$ . Then we can obtain the bounds for  $p_5$  and  $p_6$  as shown in Fig. 3.

Next, for the 3-rd layer output  $[p_7, p_8]$ :  $p_7 = p_5 + p_6$ ,  $p_8 = p_6$ , we assign  $\mathbf{A}_{(0,0,::)}^{(3,2),L} = \mathbf{A}_{(0,0,::)}^{(3,2),U} = [1, 1]$  and  $\mathbf{A}_{(0,1,::)}^{(3,2),L} = \mathbf{A}_{(0,1,::)}^{(3,2),U} = [0, 1]$  to form:

$$\begin{aligned} p_5 \cdot \mathbf{A}_{(0,0,0,0)}^{(3,2),L} + p_6 \cdot \mathbf{A}_{(0,0,0,1)}^{(3,2),L} &\leq p_7 \leq p_5 \cdot \mathbf{A}_{(0,0,0,0)}^{(3,2),U} + p_6 \cdot \mathbf{A}_{(0,0,0,1)}^{(3,2),U} \\ p_5 \cdot \mathbf{A}_{(0,1,0,0)}^{(3,2),L} + p_6 \cdot \mathbf{A}_{(0,1,0,1)}^{(3,2),L} &\leq p_8 \leq p_5 \cdot \mathbf{A}_{(0,1,0,0)}^{(3,2),U} + p_6 \cdot \mathbf{A}_{(0,1,0,1)}^{(3,2),U}. \end{aligned}$$

We can back propagate constrains to the first layer to gain final global bounds of  $p_7$  and  $p_8$ :

$$\begin{aligned} p_1 + p_2 &\leq p_7 \leq 0.5 \cdot p_1 + p_2 + 1.5 \\ 0 &\leq p_8 \leq -0.25 \cdot p_1 + 0.25 \cdot p_2 + 0.75. \end{aligned} \quad (7)$$

Similarly, we compute the global bounds for  $p_7$  and  $p_8$ .

For the multiplication layer, according to the formulations presented in Sect. 3.4, to compute the bounds of  $p_9$  in our example, we set  $a^L = l_1$ ,  $a^U = u_1$ ,  $b^L = b^U = l_7$ ,  $c^L = -l_7 \cdot l_1$ , and  $c^U = -l_7 \cdot u_1$ . Similarly, for  $p_9$ , we choose  $a^L = l_2$ ,  $a^U = u_2$ ,  $b^L = b^U = l_8$ ,  $c^L = -l_8 \cdot l_2$ , and  $c^U = -l_8 \cdot u_2$ . Thus, we get

$$-p_1 \leq p_9 \leq 2 \cdot p_7 - p_1 + 2, \quad -p_8 \leq p_{10} \leq p_8. \quad (8)$$

Instead of performing back-propagation to the input layer, we calculate the bounds for  $p_9$  and  $p_{10}$  by directly replacing  $p_7$  and  $p_8$  with Eq. 7:

$$\begin{aligned} -p_1 \leq p_9 \leq 2 \cdot p_2 + 5 \\ 0.25 \cdot p_1 - 0.25 \cdot p_2 - 0.75 \leq p_{10} \leq -0.25 \cdot p_1 + 0.25 \cdot p_2 + 0.75. \end{aligned} \quad (9)$$

Again, according to Eq. 4, we obtain  $l_9 = -2$ ,  $u_9 = 7$ ,  $l_{10} = -1$  and  $u_{10} = 1$ .

Finally, in our example,  $p_{11} = p_9$  and  $p_{12} = p_{10} - p_9$ . After propagating the linear bounds to the previous layer by replacing the  $p_9$  and  $p_{10}$  with  $p_7$  and  $p_8$  in Eq. 8, we construct the constraints of the last layer as:

$$\begin{aligned} -p_1 \leq p_{11} \leq 2 \cdot p_7 - p_1 + 2 \\ -p_8 - 2 \cdot p_7 + p_1 - 2 \leq p_{12} \leq p_8 + p_1. \end{aligned} \quad (10)$$

By substituting  $p_7$  and  $p_8$  with  $p_1$  and  $p_2$  directly, the global bounds for  $p_{11}$  and  $p_{12}$  are:

$$\begin{aligned} -p_1 \leq p_{11} \leq 2 \cdot p_2 + 5 \\ 0.25 \cdot p_1 - 2.25 \cdot p_2 - 5.75 \leq p_{12} \leq 0.75 \cdot p_1 + 0.25 \cdot p_2 + 0.75. \end{aligned} \quad (11)$$

Thereby, as described so far, the back-substitution yields  $l_{11} = -1$ ,  $u_{11} = 7$ ,  $l_{12} = -8$ ,  $u_{12} = 2.5$ , which are the final output bounds of our example network.

**Robustness Analysis:** The inputs  $p_1 = 1$  and  $p_2 = 0$  lead to output  $p_{11} = 1$  and  $p_{12} = -1$  in our example. Thus, to verify the robustness of our example network, we aim to find the maximum  $\epsilon$  that guarantees  $p_{11} \geq p_{12}$  always holds true for any perturbed inputs within the  $l_\infty$ -norm ball with a radius  $\epsilon$ . In our example, the results of  $l_{11}$ ,  $u_{11}$ ,  $l_{12}$ , and  $u_{12}$  conclude that  $p_{11} - p_{12} \in [-3.5, 3]$  which results in  $p_{11} \geq p_{12}$  failing to hold. Thus, we apply binary search to reduce the value of  $\epsilon$  and recalculate the output bounds for the network based on the new  $\epsilon$ . When the maximum iteration is reached, we stop the binary search and choose the maximum  $\epsilon$  that results in the lower bound of  $p_{11} - p_{12} \geq 0$  as the final certified distortion.

## 4 Experiments

**Dataset** We evaluate 3DVerifier on ModelNet40 (Wu et al., 2015) dataset, which contains 9843 training and 2468 testing 3D CAD models. Each CAD model is used to sample a point cloud that comprises 2048 points in three dimensions (Qi et al., 2017a). There are 40 categories of CAD models. We run verification experiments on point clouds with 1024, 512, and 64 points, and randomly selected 100 samples from all the 40 categories of the test set as the dataset to perform the experiments. All experiments are carried out on the same randomly selected dataset.

**Models** We utilize PointNet (Qi et al., 2017a) models as the 3D point cloud classifiers. Since the baseline verification method, 3DCertify (Lorenz et al., 2021), cannot handle the full PointNet with JAnet, to make a comprehensive comparison, we first perform experiments on the PointNet without JAnet. We then examine the performance of our 3DVerifier on full PointNet models. All models use the ReLU activation function.

**Baseline** (1) We choose the existing 3D robustness certifier, 3DCertify (Lorenz et al., 2021), as the main baseline for the PointNet models without JAnet, which can be viewed as a general CNN. Additionally, we also show the average distortions obtained by adversarial

attacks on 3D point cloud models that extended from the CW attack (Carlini et al., 2017; Xiang et al., 2019) and PGD attack (Kurakin et al., 2018). (2) As for the complete PointNet proposed by (Qi et al., 2017a), we provide the average and minimum distortions obtained by the CW attack for robustness estimation. The PGD attack takes a long time to seek the adversarial examples, and the attack success rate is below 10%, thus, it is not included as the comparative method.

*Implementation* The 3DVerifier is implemented via NumPy with Numba in Python. All experiments are run on a 2.10GHz Intel Xeon Platinum 8160 CPU with 512 GB memory.

## 4.1 Results for PointNet models without JAnet

In Table 1, we present the clean test accuracy (Acc.) and average certified bounds (ave) for PointNet models without JAnet. We also record the time to run one iteration of the binary search. We demonstrate that our 3DVerifier can improve upon 3DCertify in terms of run-time and tightness of bounds. To make the comparison more extensive, we train a 5-layer ( $N=5$ ) and a 9-layer ( $N=9$ ) model. The specific configurations of the model structure are presented in “Appendix F”. We also show the performance of models with different types of pooling layers: global average pooling and global maximum pooling. For experiments, we set the initial  $\epsilon$  as 0.05 and the maximum iteration of binary search as 10. The experiments are taken on three point-cloud datasets with 64, 512, and 1024 points. We can see from the results of average bounds that our 3DVerifier can obtain tighter lower bounds than 3DCertify and engages a significant gap in the distortion bounds found by CW and PGD attacks. Table 1 shows that our method is much faster than the 3DCertify. Notably, 3DVerifier enables an orders-of-magnitude improvement in efficiency for large networks. Additionally, our method can compute certified bounds of distortion constrained by  $l_1$ ,  $l_2$ , and  $l_\infty$ -norms, which is more scalable than 3DCertify in terms of norm distance.

## 4.2 Results for PointNet models with JAnet

As the 3DCertify does not include the bounds computation algorithm for multiplication operation, it can not be applied to verify the complete PointNet with JAnet architecture. Thus, as the first work to verify the point cloud models with JAnet, we show the average and minimum distortion bounds of CW attack-based method to make comparisons. We examine  $N$ -layer models ( $N=8,12,15$ ) with two types of global pooling: average pooling and maximum pooling on datasets with 64, 512, and 1024 points. The obtained certified average bounds of full PointNet models are shown in Table 2. According to previous verification works on images (e.g. (Boopathy et al., 2019)) and results in Table 1, the gap between certified bounds and attack-based average distortion is reasonable, where the average minimum distortion is ten times greater than the bounds. Thus, it reveals that our method efficiently certifies the point cloud models with JAnet in a comparable quality with models without JAnet.

## 4.3 Discussions

### 4.3.1 3DVerifier is efficient for large-scale 3D point cloud models

There are two key features that enable 3DVerifier’s efficiency for large 3D networks.

*Improved global max pooling relaxation:* the relaxation algorithm for global max pooling layer of CNN-Cert (Boopathy et al., 2019)) framework cannot be adapted directly to

**Table 1** Average certified bounds (ave) and run-time on PointNet without JAnet 4

| No.  | Pool. | Acc. | N | $l_p$      | Certified bounds |        |           |        | Our vs.       |       |        |      | Attack |  |  |
|------|-------|------|---|------------|------------------|--------|-----------|--------|---------------|-------|--------|------|--------|--|--|
|      |       |      |   |            | Our              |        | 3DCertify |        | 3DCertify (%) |       | CW     | PGD  |        |  |  |
|      |       |      |   |            | Min              | Ave    | Min       | Ave    | Time          | Ave   |        |      | Time   |  |  |
|      |       |      |   |            |                  |        |           |        |               |       |        |      |        |  |  |
| 64   | Ave   | 70.1 | 5 | $l_\infty$ | 0.0007           | 0.0122 | 0.0006    | 0.0080 | 1.6           | +52.5 | − 67.9 | 0.03 | 0.02   |  |  |
|      |       |      |   | $l_2$      | 0.0014           | 0.0433 | −         | −      | −             | −     | 0.42   | 0.53 |        |  |  |
|      |       |      |   | $l_1$      | 0.0024           | 0.0553 | −         | −      | −             | −     | 2.50   | 3.21 |        |  |  |
|      |       |      |   | $l_\infty$ | 16e−5            | 0.0044 | 12e−5     | 0.0034 | 92.1          | +29.4 | −90.4  | 0.02 | 0.02   |  |  |
|      |       |      |   | $l_2$      | 0.0014           | 0.0146 | −         | −      | −             | −     | 0.39   | 0.46 |        |  |  |
|      | Max   | 73.6 | 5 | $l_1$      | 0.0028           | 0.0188 | −         | −      | −             | −     | 2.97   | 2.55 |        |  |  |
|      |       |      |   | $l_\infty$ | 1e−4             | 0.0137 | 8e−5      | 0.0079 | 0.54          | +73.4 | −26.5  | 0.03 | 0.02   |  |  |
|      |       |      |   | $l_2$      | 0.0008           | 0.0481 | −         | −      | −             | −     | 0.16   | 0.28 |        |  |  |
|      |       |      |   | $l_1$      | 0.0021           | 0.0649 | −         | −      | −             | −     | 3.11   | 3.21 |        |  |  |
|      |       |      |   | $l_\infty$ | 0.0007           | 0.0051 | 0.0004    | 0.0036 | 51.4          | +41.7 | −83.8  | 0.02 | 0.03   |  |  |
| 512  | Ave   | 73.0 | 5 | $l_2$      | 0.0045           | 0.0209 | −         | −      | −             | −     | 0.24   | 0.16 |        |  |  |
|      |       |      |   | $l_1$      | 0.0102           | 0.0288 | −         | −      | −             | −     | 2.81   | 3.09 |        |  |  |
|      |       |      |   | $l_\infty$ | 0.0007           | 0.0104 | 0.0006    | 0.0063 | 100.4         | +15.5 | −84.8  | 0.07 | 0.02   |  |  |
|      |       |      |   | $l_2$      | 0.0079           | 0.0427 | −         | −      | −             | −     | 0.41   | 0.47 |        |  |  |
|      |       |      |   | $l_1$      | 0.0109           | 0.0648 | −         | −      | −             | −     | 4.54   | 4.41 |        |  |  |
|      | Max   | 78.3 | 5 | $l_\infty$ | 0.0007           | 0.0108 | 0.0006    | 0.0070 | 18.5          | +54.3 | −58.7  | 0.06 | 0.02   |  |  |
|      |       |      |   | $l_2$      | 0.0048           | 0.0313 | −         | −      | −             | −     | 0.26   | 0.19 |        |  |  |
|      |       |      |   | $l_1$      | 0.0116           | 0.0407 | −         | −      | −             | −     | 2.90   | 3.10 |        |  |  |
|      |       |      |   | $l_\infty$ | 0.006            | 0.0048 | *         | *      | *             | −     | 0.03   | 0.02 |        |  |  |
|      |       |      |   | $l_2$      | 0.0180           | 0.0159 | −         | −      | −             | −     | 0.26   | 0.36 |        |  |  |
| 1024 | Ave   | 72.6 | 5 | $l_1$      | 0.0632           | 0.0196 | −         | −      | −             | −     | 4.49   | 5.08 |        |  |  |
|      |       |      |   | $l_\infty$ | 0.0010           | 0.0145 | *         | *      | *             | −     | 0.05   | 0.04 |        |  |  |
|      |       |      |   | $l_2$      | 0.0125           | 0.0471 | −         | −      | −             | −     | 0.46   | 0.39 |        |  |  |
|      |       |      |   | $l_1$      | 0.0354           | 0.0544 | −         | −      | −             | −     | 2.38   | 3.02 |        |  |  |
|      |       |      |   | $l_\infty$ | 0.0006           | 0.0135 | 0.0002    | 0.0085 | 21.0          | +58.8 | −31.6  | 0.06 | 0.04   |  |  |
|      | Max   | 77.8 | 5 |            |                  |        |           |        |               |       |        |      |        |  |  |
|      |       |      |   |            |                  |        |           |        |               |       |        |      |        |  |  |
|      |       |      |   |            |                  |        |           |        |               |       |        |      |        |  |  |
|      |       |      |   |            |                  |        |           |        |               |       |        |      |        |  |  |
|      |       |      |   |            |                  |        |           |        |               |       |        |      |        |  |  |

Table 1 (continued)

| No. | Pool. | Acc. | N | $l_p$ | Certified bounds |        |      |     | 3DCertify |     |      | Our vs.       |      | Attack |      |
|-----|-------|------|---|-------|------------------|--------|------|-----|-----------|-----|------|---------------|------|--------|------|
|     |       |      |   |       | Our              |        | Time | Ave | Min       | Ave | Time | 3DCertify (%) |      | CW     | PGD  |
|     |       |      |   |       | Min              | Ave    |      |     |           |     |      | Ave           | Time |        |      |
|     |       |      |   | $l_2$ | 0.0205           | 0.0393 | 19.0 | –   | –         | –   | –    | –             | –    | 0.36   | 0.27 |
|     |       |      |   | $l_1$ | 0.0493           | 0.0500 | 18.1 | –   | –         | –   | –    | –             | –    | 3.47   | 2.45 |

For the certified bounds, the higher the better. The bounds obtained by attacks are referred as upper bounds. ‘\*’ means that computing the bounds by 3DCertify is computationally unattainable, which automatically terminates after verifying several samples. Thus, we present the results of verifying 10 samples in ‘Appendix D’

**Table 2** Average certified bounds (ave) and times on PointNet models with T-Net

| No. points | Pooling | Acc.  | N  | $l_p$      | Our   |        |        | CW    |       |
|------------|---------|-------|----|------------|-------|--------|--------|-------|-------|
|            |         |       |    |            | Ave   | Min    | Time   | Ave   | Min   |
| 64         | Average | 70.71 | 8  | $l_\infty$ | 0.341 | 0.011  | 0.12   | 0.727 | 0.021 |
|            |         |       |    | $l_2$      | 0.862 | 0.077  | 0.19   | 1.700 | 0.162 |
|            |         |       |    | $l_1$      | 1.479 | 0.136  | 0.18   | 2.049 | 0.317 |
|            |         | 80.22 | 12 | $l_\infty$ | 0.316 | 0.016  | 2.98   | 0.832 | 0.053 |
|            |         |       |    | $l_2$      | 0.588 | 0.032  | 2.86   | 2.164 | 0.276 |
|            |         |       |    | $l_1$      | 1.115 | 0.113  | 2.73   | 2.949 | 0.510 |
|            |         | 84.11 | 15 | $l_\infty$ | 0.248 | 0.012  | 23.98  | 0.792 | 0.077 |
|            |         |       |    | $l_2$      | 0.466 | 0.026  | 23.25  | 3.310 | 0.313 |
|            |         |       |    | $l_1$      | 0.857 | 0.088  | 22.79  | 4.857 | 0.875 |
|            | Max     | 74.63 | 8  | $l_\infty$ | 0.554 | 0.029  | 0.43   | 0.791 | 0.093 |
|            |         |       |    | $l_2$      | 1.118 | 0.039  | 0.41   | 2.133 | 0.137 |
|            |         |       |    | $l_1$      | 1.650 | 0.041  | 0.397  | 3.078 | 0.322 |
|            |         | 81.91 | 12 | $l_\infty$ | 0.541 | 0.007  | 4.26   | 0.846 | 0.078 |
|            |         |       |    | $l_2$      | 0.897 | 0.010  | 4.11   | 2.554 | 0.102 |
|            |         |       |    | $l_1$      | 1.326 | 0.010  | 3.18   | 3.772 | 0.167 |
|            |         | 86.77 | 15 | $l_\infty$ | 0.122 | 0.027  | 24.42  | 0.218 | 0.044 |
|            |         |       |    | $l_2$      | 0.494 | 0.030  | 24.13  | 1.285 | 0.076 |
|            |         |       |    | $l_1$      | 0.615 | 0.035  | 23.29  | 1.721 | 0.103 |
| 512        | Average | 73.53 | 12 | $l_\infty$ | 0.231 | 0.005  | 71.26  | 0.395 | 0.035 |
|            |         |       |    | $l_2$      | 1.196 | 0.027  | 68.73  | 3.106 | 0.637 |
|            |         |       |    | $l_1$      | 1.622 | 0.107  | 66.91  | 4.852 | 0.833 |
|            | Max     | 84.61 | 12 | $l_\infty$ | 0.109 | 0.0003 | 55.78  | 0.693 | 0.033 |
|            |         |       |    | $l_2$      | 0.272 | 0.0005 | 53.64  | 0.882 | 0.085 |
|            |         |       |    | $l_1$      | 0.345 | 0.043  | 50.22  | 1.175 | 0.448 |
| 1024       | Average | 71.47 | 12 | $l_\infty$ | 0.406 | 0.017  | 129.41 | 0.721 | 0.041 |
|            |         |       |    | $l_2$      | 1.142 | 0.181  | 127.88 | 1.459 | 0.342 |
|            |         |       |    | $l_1$      | 1.574 | 0.233  | 126.24 | 1.926 | 0.596 |
|            | Max     | 80.22 | 12 | $l_\infty$ | 0.374 | 0.001  | 128.22 | 0.758 | 0.005 |
|            |         |       |    | $l_2$      | 0.959 | 0.006  | 127.51 | 1.591 | 0.009 |
|            |         |       |    | $l_1$      | 1.253 | 0.009  | 120.75 | 2.163 | 0.018 |

3D point cloud models, which is computationally unattainable. Thus, we proposed a linear relaxation for the global max pooling layer based on (Singh et al., 2019). For example, to find the maximum value  $p_r = \max_{m \in \mathcal{M}}(p_m)$ , if exists  $p_j$  such that its lower bound  $l_j \geq u_k$  for all  $k \in \mathcal{M} \setminus j$ , the lower bound for the max pooling layer is  $l^r = l_j$  and upper bound is  $u^r = u_j$ . Otherwise, we set the output of the layer  $\phi^r \geq p_j$ , where  $j = \underset{m \in \mathcal{M}}{\operatorname{argmax}}(l_m)$ , and similarly  $\phi^r \leq p_k$ , where  $k = \underset{m \in \mathcal{M}}{\operatorname{argmax}}(u_m)$ . The comparison results in Table 1 indicate that implementing the improved relaxation for the max pooling layer enables 3DVerifier to compute the certified bounds much faster than the existing method, 3DCertify.

*Combining forward and backward propagation:* The verification algorithm proposed in Shi et al. (2020) evaluated the effectiveness of combining forward and backward propagation to compute the bounds for Transformer with self-attention layers. Thus, in our tool

**Table 3** Training accuracy of different structures of point cloud models

| Models                   | N  | No. trainable parameters | Accuracy (%) |
|--------------------------|----|--------------------------|--------------|
| PointNet without JAnet   | 7  | 443656                   | 72.94        |
| PointNet without JAnet   | 12 | 822472                   | 74.07        |
| Full PointNet with JAnet | 12 | 645425                   | 81.31        |

3DVerifier, we adapted the combining forward and backward propagation to compute the bounds for the multiplication layer. As Table 2 shows, the time spent for full PointNet models with JAnet is nearly the same as the time for models without JAnet.

#### 4.4 CNN-Cert can be viewed as a special case of 3DVerifier

CNN-Cert (Boopathy et al., 2019) is a general efficient framework for verifying the neural networks for image classification, employing 2D convolution layers. Although our verification method shares a similar design mechanism as CNN-Cert, our framework is superior to CNN-Cert. One key difference is the dimension of input data, PointNet 3D models adopt 1D convolution layers, which 3DVerifier can efficiently handle. Additionally, besides the general operations such as pooling, batch normalization, and convolution with ReLU activation, we can also handle models with JAnet that contains multiplication layers. Thus, 3DVerifier can tackle more complex and larger neural networks than CNN-Cert. To adapt the framework for 3D point clouds, we introduced a novel relaxation method for max-pooling layers to obtain its certified bounds, which significantly improves the verification efficiency.

### 5 Importance of JAnet

We perform an ablation study to show the importance of JAnet in the point clouds classification task. We trained three models, and recorded the number of trainable parameters for each model. The final results of training accuracy are demonstrated in Table 3. The specific layer configurations for these models are presented in “Appendix F”. As the PointNet without JAnet can be regarded as a general CNN model, while JAnet is a complex architecture that contains T-Net and multiplication layer, to interpret the effect of JAnet, we examine two models for the PointNet without JAnet: (1) 7-layer model, which abandons the JAnet in the 12-layer full PointNet; (2) 12-layer model, which intuitively adds the convolution and dense layers in T-Net to the 7-layer model. From the results, we can see that the PointNet with JAnet can improve the training accuracy significantly compared with PointNet without JAnet when they employ the same number of layers. Therefore, JAnet plays an important role in improving the performance of point clouds models.

### 6 Conclusion

In this paper, we proposed an efficient and general robustness verification framework to certify the robustness of 3D point cloud models. By employing the linear relaxation for the multiplication layer and combining forward and backward propagation, we can efficiently compute

the certified bounds for various model architectures such as convolution, global pooling, batch normalization, and multiplication. Our experimental results on different models and point clouds with a different number of points confirm the superiority of 3DVerifier in terms of both computational efficiency and tightness of the certified bounds.

## Appendix A Relaxations for non-linearity

In Sect. 3.3, we mentioned that to compute bounds for nonlinear functions, we choose appropriate parameters to form the linear relaxation to bound the layer containing non-linear operations. Suppose that the non-linear function is  $f(y)$ , where  $y$  is the output from previous layer  $\Phi(\mathbf{P})^{l-1}$ . The bound obtained from the previous layer is  $[l, u]$ . Thus, our goal is to bound the  $f(y)$  by the following constraints:  $\alpha^{l,L}y + \beta^{l,L} \leq f(y) \leq \alpha^{l,U}y + \beta^{l,U}$ , where parameters  $\alpha^{l,L}$ ,  $\beta^{l,L}$ ,  $\alpha^{l,U}$ , and  $\beta^{l,U}$  are chosen dependent on the lower and upper bound,  $l$  and  $u$ , of the previous layer, which follow different rules for different functions.

The way to bound the nonlinear functions has been thoroughly discussed in previous works on images (e.g. (Zhang et al., 2018; Boopathy et al., 2019; Singh et al., 2019)). Thus, by reviewing their methods, we synthesise our relaxations for the nonlinear functions to determine the parameters according to  $l$  and  $u$ .

**ReLU** The most common activation function is the ReLU function, which is represented as  $f(y) = \max(0, y)$ . Thus, if  $u \leq 0$ , the output of  $f(y)$  is 0; if  $l \geq 0$ , the output will exactly be  $y$ . As for the situation that  $l < 0$  and  $u > 0$ , we set the upper bound  $u$  to be the line crossing two endpoints:  $(l, 0)$  and  $(u, f(y))$ , which can be represented as  $f^U(y) = \frac{u(y-l)}{(u-l)}$ . As for the expression for the lower bound, to make the bounds tighter, we consider two cases: if  $u > |l|$ ,  $f^L(y) = y$  and otherwise  $f^L(y) = y$ . Therefore, we can choose the  $\alpha^U = \frac{u}{(u-l)}$ ,  $\beta^U = \frac{-ul}{(u-l)}$ ,  $\beta^L = 0$ ;  $\alpha^L = 0$  when  $u < |l|$ , and otherwise  $\alpha^L = 1$ .

## Appendix B Derivation of the linear functions to bound the multiplication layer

The mathematical proof to choose the optimal parameters of the linear functions to bound the multiplication layer is evaluated in Shi et al. (2020). Here we will present how to derive the optimisation problem based on the derivation in Shi et al. (2020). The goal of the linear relaxation is to bound the multiplication on two inputs,  $p_1 p_2$ , which can be represented by the following constraints:

$$a^L p_1 + b^L p_2 + c^L \leq p_1 p_2 \leq a^U p_1 + b^U p_2 + c^U \quad (12)$$

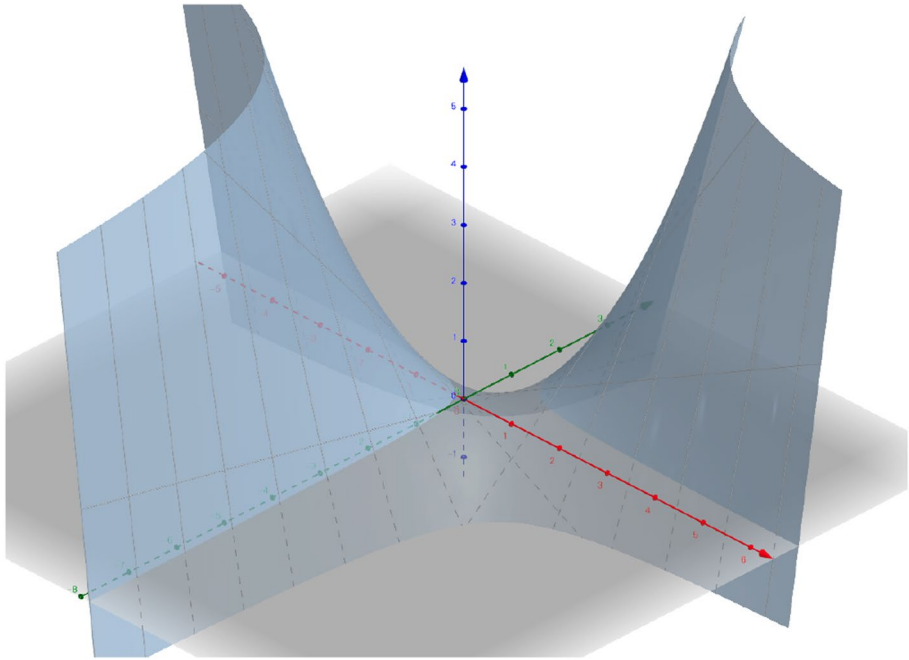
As we have already obtained the bounds of  $p_1$  and  $p_2$ , given the bounds of  $p_1$  is  $[l_1, u_1]$  and for  $p_2$  is  $[l_2, u_2]$ , we can choose the appropriate parameters of  $a^L$ ,  $a^U$ ,  $b^L$ ,  $b^U$ ,  $c^L$  and  $c^U$  according to the computed bounds.

### B.1 Derivation for the lower bound

Let the objective function be :

$$G^L(p_1, p_2) = p_1 p_2 - (a^L p_1 + b^L p_2 + c^L) \quad (13)$$





**Fig. 4** Saddle surface

To find the appropriate parameters, we need find the optimal value of  $a^L$ ,  $b^L$  and  $c^L$  to minimize the gap function  $G$ ,  $\min G^L(p_1, p_2)$ . As the  $p_1$  and  $p_2$  are constrained by the area  $[l_1, u_1] \times [l_2, u_2]$ . The objective function can be reformulated as:

$$\max_{a^L, b^L, c^L} F^L(p_1, p_2) = \max_{a^L, b^L, c^L} \int_{p_1 \in [l_1, u_1]} \int_{p_2 \in [l_2, u_2]} a^L p_1 + b^L p_2 + c^L \quad (14)$$

s.t.  $G^L(p_1, p_2) \geq 0$  ( $\forall (p_1, p_2) \in [l_1, u_1] \times [l_2, u_2]$ ).

First, we need to ensure that  $G^L(p_1, p_2) \geq 0$  by finding the minimum value of  $G(p_1, p_2)$ . The first-order partial derivatives of  $G^L$  are:  $\frac{\partial G^L}{\partial p_1} = p_2 - a^L$ ,  $\frac{\partial G^L}{\partial p_2} = p_1 - b^L$ . The critical points are  $\frac{\partial G^L}{\partial p_1} = 0$  and  $\frac{\partial G^L}{\partial p_2} = 0$ , which leads to  $p_2 = a^L$  and  $p_1 = b^L$ . Then we form the Hessian matrix to justify the behavior of the critical points:

$$H(G^L(p_1, p_2)) = \begin{pmatrix} \frac{\partial^2 G^L}{\partial p_1^2} & \frac{\partial^2 G^L}{\partial p_1 \partial p_2} \\ \frac{\partial^2 G^L}{\partial p_2 \partial p_1} & \frac{\partial^2 G^L}{\partial p_2^2} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (15)$$

Then we compute the Hessian:

$$\det H(G^L(p_1, p_2)) = \left( \frac{\partial^2 G^L}{\partial p_1^2} \right) \left( \frac{\partial^2 G^L}{\partial p_2^2} \right) - \frac{\partial^2 G^L}{\partial p_1 \partial p_2} = -1 < 0 \quad (16)$$

Thus, the critical points are saddle points, which can be illustrated in Fig. 4. Thus, the minimum value of  $G^L(p_1, p_2)$  can be found on the boundary. As there exists a point  $(p_1^0, p_2^0) \in [l_1, u_1] \times [l_2, u_2]$  such that  $G^L(p_1^0, p_2^0) = 0$ . Thereby, we need to satisfy constraints on  $G^L(p_1, p_2)$  that  $G^L(p_1^0, p_2^0) = 0$  and the value of  $G^L$  on boundary points:  $G^L(l_1, u_1), G^L(l_1, u_2), G^L(l_2, u_1), G^L(l_2, u_2) \geq 0$ . Equivalently, this can be formulated as:

$$\begin{cases} c = p_1^0 p_2^0 - a^L p_1^0 - b^L p_2^0 \\ l_1 l_2 - a^L (l_1 - p_1^0) - b^L (l_1 - p_2^0) - p_1^0 p_2^0 \geq 0 \\ l_1 u_2 - a^L (l_1 - p_1^0) - b^L (u_2 - p_2^0) - p_1^0 p_2^0 \geq 0 \\ u_1 l_2 - a^L (u_1 - p_1^0) - b^L (l_2 - p_2^0) - p_1^0 p_2^0 \geq 0 \\ u_1 u_2 - a^L (u_1 - p_1^0) - b^L (u_2 - p_2^0) - p_1^0 p_2^0 \geq 0 \end{cases} \quad (17)$$

By substituting the  $c^L$  in Eq. 13 with equation Eq. 17, we can obtain:

$$F^L(p_1, p_2) = \Lambda[(l_1 + u_1 - 2p_1^0)a^L + (l_2 + u_2 - 2p_2^0)b^L + 2p_1^0 p_2^0], \quad (18)$$

where  $\Lambda = \frac{(u_1 - l_1)(u_2 - l_2)}{2}$ . As the minimum value can only be found on the boundary, the  $p_1^0$  can be  $l_1$  or  $u_1$ .

- If  $p_1^0 = l_1$ : By substituting it in Eqs. 17 and 18, we get:

$$\begin{cases} a^L \leq \frac{u_1 l_2 - l_1 p_2^0 - b^L (l_2 - p_2^0)}{u_1 - l_1}, a^L \leq \frac{u_1 u_2 - l_1 p_2^0 - b^L (u_2 - p_2^0)}{u_1 - l_1}, l_1 \leq b^L \leq l_1 \Leftrightarrow b^L = l_1. \\ F^L(p_1, p_2) = \Lambda[(u_1 - l_1)a^L + (l_2 + u_2 - 2p_2^0)b^L + 2l_1 p_2^0]. \end{cases} \quad (19)$$

Next, replacing  $b^L = l_1$ , we can obtain:

$$\begin{cases} F^L(p_1, p_2) = \Lambda[(u_1 - l_1)a^L + l_1(l_2 + u_2)], \\ (u_1 - l_1)a^L \leq -l_1 p_2^0 + \min(u_1 l_2 - b^L(l_2 - p_2^0), u_1 u_2 - b^L(u_2 - p_2^0)) = (u_1 - l_1)l_2. \end{cases}$$

As a result, we obtain  $a^L \leq l_2$ . To maximize  $F^L(p_1, p_2)$ , because  $\Lambda \geq 0$ , we adapt  $a^L = l_2$  to get  $F^L(p_1, p_2) = \Lambda(u_1 l_2 + l_1 u_2)$ , which is constant.

- If  $p_1^0 = u_1$ : Similarly, we can compute

$$\begin{cases} a^L \geq \frac{l_1 l_2 - u_1 p_2^0 - b^L (l_2 - p_2^0)}{l_1 - u_1}, a^L \geq \frac{l_1 u_2 - u_1 p_2^0 - b^L (u_2 - p_2^0)}{l_1 - u_1}, u_1 \leq b^L \leq u_1 \Leftrightarrow b^L = u_1. \\ F^L(p_1, p_2) = \Lambda[(l_1 - u_1)a^L + (l_2 + u_2 - 2p_2^0)b^L + 2u_1 p_2^0] \end{cases}$$

By replacing  $b^L = u_1$ , we obtain

$$\begin{cases} F^L(p_1, p_2) = \Lambda[(l_1 - u_1)a^L + u_1(l_2 + u_2)], \\ (l_1 - u_1)a^L \leq -u_1 p_2^0 + \min(l_1 l_2 - b^L(l_2 - p_2^0), l_1 u_2 - b^L(u_2 - p_2^0)) = (l_1 - u_1)u_2 \end{cases}$$

Then, we conclude that  $a^L \geq u_2$ , and  $F^L = \Lambda(l_1 u_2 + u_1 l_2)$

As the  $F^L$  are the same for above two cases, which is not determined by  $p_2^0$ , we can choose  $p_2^0 = l_2$ . Thus we can obtain the parameters to compute the lower bounds:  $a^L = l_2, b^L = l_1, c^L = -l_1 l_2$

## B.2 Derivation for the upper bound

Similarly, to compute the upper bound, we can formulate the  $G^U(p_1, p_2)$  as:

$$G^U(p_1, p_2) = (a^U p_1 + b^U p_2 + c^U) - p_1 p_2 \quad (20)$$

Thus, the objective function to compute the upper bound is:

$$\min_{a^U, b^U, c^U} F^U(p_1, p_2) = \min_{a^U, b^U, c^U} \int_{p_1 \in [l_1, u_1]} \int_{p_2 \in [l_2, u_2]} a^U p_1 + b^U p_2 + c^U, \quad (21)$$

and then it can be expressed as:

$$F^U(p_1, p_2) = \Lambda[(l_1 + u_1 - 2p_1^0)a^U + (l_2 + u_2 - 2p_2^0)b^U + 2p_1^0 p_2^0], \Lambda = \frac{(u_1 - l_1)(u_2 - l_2)}{2}$$

- If we set  $p_1^0 = l_1$ :

$$\begin{cases} a^U \geq \frac{u_1 l_2 - l_1 p_2^0 - b^U (l_2 - p_2^0)}{u_1 - l_1}, a^U \leq \frac{u_1 u_2 - l_1 p_2^0 - b^U (u_2 - p_2^0)}{u_1 - l_1}, l_1 \leq b^U \leq l_1 \Leftrightarrow b^U = l_1. \\ F^U(p_1, p_2) = \Lambda[(u_1 - l_1)a^U + (l_2 + u_2 - 2p_2^0)b^U + 2l_1 p_2^0] \end{cases}$$

After substituting  $b^U = l_1$ , we get

$$\begin{cases} F^U(p_1, p_2) = \Lambda[(u_1 - l_1)a^U + l_1(l_2 + u_2)], \\ (u_1 - l_1)a^U \geq -l_1 p_2^0 + \max(u_1 l_2 - b^U(l_2 - p_2^0), u_1 u_2 - b^U(u_2 - p_2^0)) = (u_1 - l_1)u_2 \end{cases}$$

Thus,  $a^U \geq u_2$ . To minimize  $F^U(p_1, p_2)$ , we adapt  $a^U = u_2$  to get  $F^U(p_1, p_2) = \Lambda(l_1 l_2 + u_1 u_2)$ .

- If we set  $p_1^0 = l_1$ :

$$\begin{cases} a^U \leq \frac{l_1 l_2 - u_1 p_2^0 - b^U (l_2 - p_2^0)}{l_1 - u_1}, a^U \leq \frac{l_1 u_2 - u_1 p_2^0 - b^U (u_2 - p_2^0)}{l_1 - u_1}, u_1 \leq b^U \leq u_1 \Leftrightarrow b^U = u_1 \\ F^U(p_1, p_2) = \Lambda[(l_1 - u_1)a^U + (l_2 + u_2 - 2p_2^0)b^U + 2u_1 p_2^0] \end{cases}$$

Next, we get

$$\begin{cases} F^U(p_1, p_2) = \Lambda[(l_1 - u_1)a^U + u_1(l_2 + u_2)], \\ (l_1 - u_1)a^U \leq -u_1 p_2^0 + \max(l_1 l_2 - b^U(l_2 - p_2^0), l_1 u_2 - b^U(u_2 - p_2^0)) = (l_1 - u_1)l_2 \end{cases}$$

Thus, we can conclude that  $a^U \leq l_2$ , and to minimize  $F^U$ , we set  $a^U = l_2$  to form  $F^U = \Lambda(l_1 l_2 + u_1 u_2)$ .

As the two results for  $F^U$  are the same, we choose  $p_2^0 = l_2$  to obtain the final parameters for the upper bound:  $a^U = u_2, b^U = l_1, c^U = -l_1 u_2$ .

## Appendix C Extra experiments

As computing the bounds by 3DCertify is computationally unattainable, it will terminate after verifying several samples. Thus, we present the results to verify ten samples. From the Table 4, we can see that our method could obtain tighter bounds and significantly reduce the time.

**Table 4** Average bounds and computational times on PointNet models without JAnet

| No.  | Pool. | Acc. | N | $l_p$        | Certified bounds |        |           |        | Our vs.       |        |       |       | Attack |       |
|------|-------|------|---|--------------|------------------|--------|-----------|--------|---------------|--------|-------|-------|--------|-------|
|      |       |      |   |              | Our              |        | 3DCertify |        | 3DCertify (%) |        | CW    |       | PGD    |       |
|      |       |      |   |              | Min              | Ave    | Min       | Ave    | Min           | Time   | Ave   | Time  | Ave    | Ave   |
| 512  | Max   | 82.1 | 9 | $l_\infty$   | 0.0007           | 0.0052 | 0.0002    | 0.0030 | 487.2         | −88.0  | 0.033 | 0.017 | 0.033  | 0.017 |
|      |       |      |   | $l_2$        | 0.0012           | 0.0159 | −         | −      | −             | −      | 0.263 | 0.358 | 0.263  | 0.358 |
|      |       |      |   | $l_1$        | 0.0019           | 0.0196 | −         | −      | −             | −      | 4.492 | 5.079 | 4.492  | 5.079 |
| 1024 | Ave.  | 72.6 | 5 | $l_\infty^*$ | 0.001            | 0.0141 | 0.0007    | 0.0107 | 472.15        | +31.77 | 0.051 | 0.039 | 0.051  | 0.039 |
|      |       |      |   | $l_2$        | 0.008            | 0.0471 | −         | −      | −             | −      | 0.464 | 0.388 | 0.464  | 0.388 |
|      |       |      |   | $l_1$        | 0.015            | 0.0544 | −         | −      | −             | −      | 2.38  | 3.015 | 2.38   | 3.015 |

For the certified bounds, the higher, the better. The bounds obtained by attack based method can be viewed as upper bounds

## Appendix D Increasing number of test samples

In Table 5 we present the results for verifying 1000 samples on 64 points. The results show that the obtained average certified bounds and run time for one epoch are similar to those under 100 samples. As the average certified bound is more related to the threat model, 100 samples are enough to evaluate the average certified bound, as they are randomly selected from different categories.

**Table 5** Average certified bounds (ave) and run-time on PointNet without JANet on 1000 samples

| Pooling | Acc.  | N | $l_p$      | Certified bounds |        |       |           |        |       | Our vs.       |        |
|---------|-------|---|------------|------------------|--------|-------|-----------|--------|-------|---------------|--------|
|         |       |   |            | Our              |        |       | 3DCertify |        |       | 3DCertify (%) |        |
|         |       |   |            | Min              | Ave    | Time  | Min       | Ave    | Time  | Ave           | Time   |
| Ave     | 70.1  | 5 | $l_\infty$ | 0.00001          | 0.0122 | 0.77  | 0.000006  | 0.0077 | 1.34  | +58.44        | −42.5  |
|         |       |   | $l_2$      | 0.00007          | 0.0437 | 0.67  | –         | –      | –     | –             | –      |
|         |       |   | $l_1$      | 0.00025          | 0.0562 | 0.81  | –         | –      | –     | –             | –      |
|         | 79.25 | 9 | $l_\infty$ | 0.00002          | 0.0045 | 10.87 | 0.00001   | 0.0033 | 80.5  | +36.36        | −86.50 |
|         |       |   | $l_2$      | 0.0001           | 0.0146 | 9.85  | –         | –      | –     | –             | –      |
|         |       |   | $l_1$      | 0.00022          | 1.0187 | 9.43  | –         | –      | –     | –             | –      |
|         | 73.55 | 5 | $l_\infty$ | 0.00001          | 0.0142 | 0.44  | 0.000007  | 0.0078 | 0.552 | +82.05        | −20.3  |
|         |       |   | $l_2$      | 0.00009          | 0.0494 | 1.03  | –         | –      | –     | –             | –      |
|         |       |   | $l_1$      | 0.00021          | 0.0666 | 0.84  | –         | –      | –     | –             | –      |
| Max     | 81.22 | 9 | $l_\infty$ | 0.00001          | 0.0050 | 12.73 | 0.00001   | 0.0036 | 50.22 | +38.89        | −74.65 |
|         |       |   | $l_2$      | 0.00009          | 0.0210 | 11.77 | –         | –      | –     | –             | –      |
|         |       |   | $l_1$      | 0.0002           | 0.0289 | 10.43 | –         | –      | –     | –             | –      |

For the certified bounds, the higher, the better. The bounds obtained by attacks are referred as upper bounds

## Appendix E Experiments on extra datasets

As our method is to certify the point cloud models, any point cloud datasets can be used to certify. We have already show our method's generalization on different point cloud models, in this section we will show its generalization on different point cloud models. The most popular data set that used in training PointNet and certify point cloud models is the ModelNet40, which is presented in the main text. Another popular point cloud dataset is ModelNet10, which is also collected from 3D CAD models, and are a very clean dataset, where 10 most popular object categories are manually selected and are then manually aligned the orientation of the CAD models for this 10-class subset. Moreover, we also present the experiments in full PointNet using our method on SydneyUrban dataset, which contains various common road objects scanned with a Velodyne HDL-64E LIDAR. There are 631 scans of objects in total that contains across categories of signs, vehicles, pedestrians, signs and trees, which can be visualized in Fig. 5

We use the ModelNet10 on 64 points to compare our work and baseline work on 64 points. The results shown in Table 6 confirms that our work still engage tighter bound and less run-time. We perform our method on ModelNet10 and SydneyUrban dataset on 64 points with an 8-layer full PointNet model, and the results are demonstrated in Table 7.

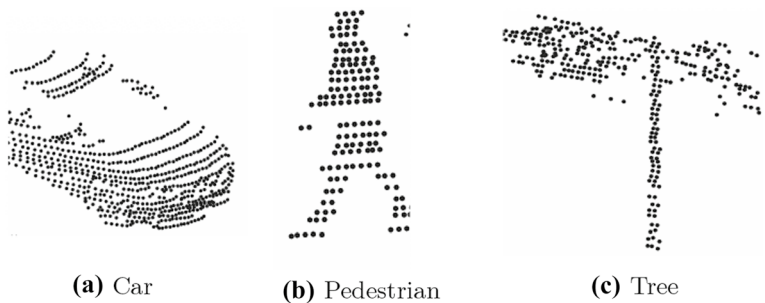
**Table 6** Average certified bounds (ave) and run-time on PointNet without JANet

| Pooling | Acc. | N | $l_p$      | Certified bounds |        |      |           |        |      | Our vs.       |        |
|---------|------|---|------------|------------------|--------|------|-----------|--------|------|---------------|--------|
|         |      |   |            | Our              |        |      | 3DCertify |        |      | 3DCertify (%) |        |
|         |      |   |            | Min              | Ave    | Time | Min       | Ave    | Time | Ave           | Time   |
| Ave     | 75.1 | 5 | $l_\infty$ | 0.00176          | 0.0139 | 0.93 | 0.00113   | 0.0084 | 1.51 | +55.75        | −38.41 |
|         |      |   | $l_2$      | 0.0132           | 0.0611 | 1.01 | –         | –      | –    | –             | –      |
|         |      |   | $l_1$      | 0.0233           | 0.0788 | 1.22 | –         | –      | –    | –             | –      |
| Max     | 78.2 | 5 | $l_\infty$ | 0.00083          | 0.0215 | 0.66 | 0.00077   | 0.0177 | 0.71 | +21.5         | −7.04  |
|         |      |   | $l_2$      | 0.0044           | 0.0680 | 0.95 | –         | –      | –    | –             | –      |
|         |      |   | $l_1$      | 0.0061           | 0.0903 | 1.22 | –         | –      | –    | –             | –      |

For the certified bounds, the higher, the better. The bounds obtained by attacks are referred as upper bounds for ModelNet10 with 64 points

**Table 7** Average certified bounds (ave) and times on PointNet models with T-Net for ModelNet10 and Sydney Urban dataset with 64 points

| Dataset    | Pooling | Acc. | N | $l_p$      | Our   |        |      | CW    |       |
|------------|---------|------|---|------------|-------|--------|------|-------|-------|
|            |         |      |   |            | Ave   | Min    | Time | Ave   | Min   |
| ModelNet10 | Average | 72.1 | 8 | $l_\infty$ | 0.451 | 0.01   | 0.14 | 0.931 | 0.022 |
|            |         |      |   | $l_2$      | 1.29  | 0.07   | 0.27 | 1.992 | 0.141 |
|            |         |      |   | $l_1$      | 2.05  | 0.13   | 0.18 | 2.591 | 0.422 |
|            | Max     | 78.2 | 8 | $l_\infty$ | 0.535 | 0.01   | 0.31 | 0.621 | 0.067 |
|            |         |      |   | $l_2$      | 1.534 | 0.04   | 0.34 | 2.773 | 0.152 |
|            |         |      |   | $l_1$      | 1.821 | 0.07   | 0.31 | 3.266 | 0.411 |
| Sydney     | Average | 62.1 | 8 | $l_\infty$ | 0.668 | 0.010  | 0.18 | 1.225 | 0.018 |
|            |         |      |   | $l_2$      | 1.66  | 0.020  | 0.16 | 2.830 | 0.132 |
|            |         |      |   | $l_1$      | 1.993 | 0.020  | 0.10 | 3.744 | 0.210 |
|            | Max     | 67.4 | 8 | $l_\infty$ | 0.497 | 0.0002 | 0.11 | 0.843 | 0.072 |
|            |         |      |   | $l_2$      | 0.685 | 0.0003 | 0.08 | 2.771 | 0.152 |
|            |         |      |   | $l_1$      | 0.968 | 0.0004 | 0.10 | 1.439 | 0.334 |



**Fig. 5** Visualization of Sydney Urban dataset

## Appendix F Configuration of various PointNet models

We present the specific layer configurations for 13-layer full PointNet with JANet in Tables 8, 9, 10, and 11 PointNets without JANet in Tables 12, 13, 14, and 15.

**Table 8** Configuration for T-Net

| No | Type             | No. features | Normalization      | Activation function |
|----|------------------|--------------|--------------------|---------------------|
| 1  | Conv1D           | 32           | BatchNormalization | ReLU                |
| 2  | Conv1D           | 64           | BatchNormalization | ReLU                |
| 3  | Conv1D           | 512          | BatchNormalization | ReLU                |
| 4  | GlobalMaxpooling |              |                    |                     |
| 5  | Dense            | 256          | BatchNormalization | ReLU                |
| 6  | Dense            | 512          | BatchNormalization | ReLU                |
|    | Reshape          |              |                    |                     |
|    | Multiplication   |              |                    |                     |

**Table 9** Configuration for full PointNet with JANet (N=12)

| No | Type             | No. features | normalization      | activation function |
|----|------------------|--------------|--------------------|---------------------|
|    | T-Net            |              |                    |                     |
| 7  | Conv1D           | 32           | BatchNormalization | ReLU                |
| 8  | Conv1D           | 64           | BatchNormalization | ReLU                |
| 9  | Conv1D           | 512          | BatchNormalization | ReLU                |
| 10 | GlobalMaxpooling |              |                    |                     |
| 11 | Dense            | 512          | BatchNormalization | ReLU                |
| 12 | Dense            | 256          | BatchNormalization | ReLU                |
|    | Dropout          |              |                    |                     |

**Table 10** Configuration for small PointNet with JANet (N=8)

| No | Type                                          | No. features | Normalization      | Activation function |
|----|-----------------------------------------------|--------------|--------------------|---------------------|
| 1  | Conv1D                                        | 32           | BatchNormalization | ReLU                |
| 2  | Conv1D                                        | 64           | BatchNormalization | ReLU                |
| 3  | Conv1D                                        | 9            | BatchNormalization | ReLU                |
| 4  | GlobalMaxpooling<br>Reshape<br>Multiplication |              |                    |                     |
| 5  | Conv1D                                        | 32           | BatchNormalization | ReLU                |
| 6  | Conv1D                                        | 64           | BatchNormalization | ReLU                |
| 7  | GlobalMaxpooling                              |              |                    |                     |
| 8  | Dense<br>Dropout                              | 128          | BatchNormalization | ReLU                |

**Table 11** Configuration for full PointNet with JANet (N=15)

| No | Type             | No. features | Normalization      | Activation function |
|----|------------------|--------------|--------------------|---------------------|
|    | T-Net            |              |                    |                     |
| 7  | Conv1D           | 32           | BatchNormalization | ReLU                |
| 8  | Conv1D           | 64           | BatchNormalization | ReLU                |
| 9  | Conv1D           | 512          | BatchNormalization | ReLU                |
| 10 | Conv1D           | 1024         | BatchNormalization | ReLU                |
| 11 | GlobalMaxpooling |              |                    |                     |
| 12 | Dense            | 1024         | BatchNormalization | ReLU                |
| 13 | Dense            | 512          | BatchNormalization | ReLU                |
| 14 | Dense<br>Dropout | 512          | BatchNormalization | ReLU                |
| 15 | Dense            | 128          | BatchNormalization | ReLU                |

**Table 12** Configuration for 5-layer PointNet without JANet (N=5)

| No | Type             | No. features | Normalization      | Activation function |
|----|------------------|--------------|--------------------|---------------------|
| 1  | Conv1D           | 64           | BatchNormalization | ReLU                |
| 2  | Conv1D           | 256          | BatchNormalization | ReLU                |
| 3  | GlobalMaxpooling |              |                    |                     |
| 4  | Dense            | 256          | BatchNormalization | ReLU                |
| 5  | Dense<br>Dropout | 128          | BatchNormalization | ReLU                |



**Table 13** Configuration for 7-layer PointNet without JAnet (N=7)

| No | Type             | No. features | Normalization      | Activa-<br>tion<br>function |
|----|------------------|--------------|--------------------|-----------------------------|
| 1  | Conv1D           | 32           | BatchNormalization | ReLU                        |
| 2  | Conv1D           | 32           | BatchNormalization | ReLU                        |
| 3  | Conv1D           | 64           | BatchNormalization | ReLU                        |
| 4  | Conv1D           | 512          | BatchNormalization | ReLU                        |
| 5  | GlobalMaxpooling |              |                    |                             |
| 6  | Dense            | 512          | BatchNormalization | ReLU                        |
| 7  | Dense            | 256          | BatchNormalization | ReLU                        |
|    | Dropout          |              |                    |                             |

**Table 14** Configuration for 9-layer PointNet without JAnet (N=9)

| No | Type             | No. features | Normalization      | Activa-<br>tion<br>function |
|----|------------------|--------------|--------------------|-----------------------------|
| 1  | Conv1D           | 64           | BatchNormalization | ReLU                        |
| 2  | Conv1D           | 64           | BatchNormalization | ReLU                        |
| 3  | Conv1D           | 64           | BatchNormalization | ReLU                        |
| 4  | Conv1D           | 128          | BatchNormalization | ReLU                        |
| 5  | Conv1D           | 1024         | BatchNormalization | ReLU                        |
| 6  | GlobalMaxpooling |              |                    |                             |
| 7  | Dense            | 512          | BatchNormalization | ReLU                        |
| 8  | Dense            | 256          | BatchNormalization | ReLU                        |
| 9  | Linear           |              |                    |                             |
|    | Dropout          |              |                    |                             |

**Table 15** Configuration for 12-layer PointNet without JANet (N=12)

| No | Type             | No. features | Normalization      | Activation function |
|----|------------------|--------------|--------------------|---------------------|
| 1  | Conv1D           | 32           | BatchNormalization | ReLU                |
| 2  | Conv1D           | 32           | BatchNormalization | ReLU                |
| 3  | Conv1D           | 64           | BatchNormalization | ReLU                |
| 4  | Conv1D           | 64           | BatchNormalization | ReLU                |
| 5  | Conv1D           | 512          | BatchNormalization | ReLU                |
| 6  | Conv1D           | 512          | BatchNormalization | ReLU                |
| 7  | GlobalMaxpooling |              |                    |                     |
| 8  | Dense            | 512          | BatchNormalization | ReLU                |
| 9  | Dense            | 256          | BatchNormalization | ReLU                |
| 10 | Dense            | 256          | BatchNormalization | ReLU                |
| 11 | Dense            | 128          | BatchNormalization | ReLU                |
| 12 | Dense            | 128          | BatchNormalization | ReLU                |
|    | Dropout          |              |                    |                     |

**Author Contributions** RM contributed to the idea, algorithm, theoretical analysis, writing, and experiments. RM conducted this research when she was a visiting PhD student at the University of Exeter. WR contributed to the idea, theoretical analysis, and writing. LM contributed to the theoretical analysis and writing. QN contributed to the writing.

**Funding** This work is supported by Partnership Resource Fund on EPSRC project on ORCA Hub [EP/R026173/1]. This work was also funded by the Faculty of Science and Technology of Lancaster University. We also thank the High-End Computing facility at Lancaster University for the computing resources.

## Declarations

**Conflict of interest** The authors declare that they have no conflict of interest

**Ethics approval and Consent to participate/publication** Not applicable.

**Code/data availability** Our code is available on <https://github.com/TrustAI/3DVerifier>.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

## References

- Aoki, Y., Goforth, H., & Srivatsan, R. A., et al. (2019). Pointnetlk: Robust & efficient point cloud registration using pointnet. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7163–7172.

- Athalye, A., Carlini, N., & Wagner, D. (2018). Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *International Conference on Machine Learning*, PMLR, pp. 274–283.
- Boopathy, A., Weng, T. W., & Chen, P. Y., et al. (2019). Cnn-cert: An efficient framework for certifying robustness of convolutional neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 3240–3247.
- Bunel, R. R., Turkaslan, I., & Torr, P., et al. (2018). A unified view of piecewise linear neural network verification. In *Proceedings of Neural Information Processing Systems*, pp. 4795–4804.
- Cao, Y., Xiao, C., & Yang, D., et al. (2019). Adversarial objects against lidar-based autonomous driving systems. [arXiv:1907.05418](https://arxiv.org/abs/1907.05418).
- Carlini, N., & Wagner, D. (2017). Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy (sp)*. IEEE, pp. 39–57.
- Chen, X., Ma, H., & Wan, J., et al. (2017). Multi-view 3D object detection network for autonomous driving. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1907–1915.
- Chen, X., Jiang, K., Zhu, Y., et al. (2021). Individual tree crown segmentation directly from uav-borne lidar data using the pointnet of deep learning. *Forests*, 12(2), 131.
- Dvijotham, K., Stanforth, R., & Gowal, S., et al. (2018). A dual approach to scalable verification of deep networks. In *UAI*, p. 3.
- Gehr, T., Mirman, M., & Drachler-Cohen, D., et al. (2018). Ai2: Safety and robustness certification of neural networks with abstract interpretation. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 3–18.
- Goodfellow, I. J., Shlens, J., & Szegedy, C. (2014). Explaining and harnessing adversarial examples. [arXiv:1412.6572](https://arxiv.org/abs/1412.6572).
- Jin, G., Yi, X., Zhang, L., et al. (2020). How does weight correlation affect generalisation ability of deep neural networks? *Advances in Neural Information Processing Systems*, 33, 21,346–21,356.
- Jin, G., Yi, X., & Huang, W., et al. (2022). Enhancing adversarial training with second-order statistics of weights. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15,273–15,283.
- Katz, G., Barrett, C., & Dill, D. L., et al. (2017). Reluplex: An efficient smt solver for verifying deep neural networks. In *International Conference on Computer Aided Verification*. Springer, pp. 97–117.
- Kurakin, A., Goodfellow, I. J., & Bengio, S. (2018). Adversarial examples in the physical world. In *Artificial Intelligence Safety and Security*. Chapman and Hall/CRC, pp. 99–112.
- Lee, K., Chen, Z., & Yan, X., et al. (2020). Shapeadv: Generating shape-aware adversarial 3d point clouds. [arXiv:2005.11626](https://arxiv.org/abs/2005.11626).
- Liang, M., Yang, B., & Wang, S., et al. (2018). Deep continuous fusion for multi-sensor 3d object detection. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 641–656.
- Liu, D., Yu, R., & Su, H. (2019). Extending adversarial attacks and defenses to deep 3d point cloud classifiers. In *2019 IEEE International Conference on Image Processing (ICIP)*. IEEE, pp. 2279–2283.
- Lorenz, T., Ruoss, A., & Balunović, M., et al. (2021). Robustness certification for point cloud models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 7608–7618.
- Mu, R., Ruan, W., & Marcolino, L. S., et al. (2021). Sparse adversarial video attacks with spatial transformations. [arXiv:2111.05468](https://arxiv.org/abs/2111.05468).
- Paigwar, A., Erkent, O., & Wolf, C., et al. (2019). Attentional pointnet for 3d-object detection in point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops* 0–0.
- Qi, C. R., Su, H., & Mo, K., et al. (2017a). PointNet: Deep learning on point sets for 3D classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 652–660.
- Qi, C. R., Yi, L., & Su, H., et al. (2017b). PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Proceedings of Neural Information Processing Systems*, pp. 5105–5114.
- Salman, H., Yang, G., & Zhang, H., et al. (2019). A convex relaxation barrier to tight robustness verification of neural networks. In *Advances in Neural Information Processing Systems*, pp. 9835–9846.
- Shi, Z., Zhang, H., & Chang, K. W., et al. (2020). Robustness verification for transformers. [arXiv:2002.06622](https://arxiv.org/abs/2002.06622).
- Singh, G., Gehr, T., & Püschel, M., et al. (2019). An abstract domain for certifying neural networks. In *Proceedings of the ACM on Programming Languages* 3(POPL), pp. 1–30.
- Stets, J. D., Sun, Y., & Corning, W., et al. (2017). Visualization and labeling of point clouds in virtual reality. In *SIGGRAPH Asia 2017 Posters*, pp. 1–2.
- Sun, J., Koenig, K., & Cao, Y., et al. (2020). On adversarial robustness of 3D point cloud classification under adaptive attacks. [arXiv:2011.11922](https://arxiv.org/abs/2011.11922).
- Szegedy, C., Zaremba, W., & Sutskever, I., et al. (2013). Intriguing properties of neural networks. [arXiv:1312.6199](https://arxiv.org/abs/1312.6199).

- Tjeng, V., Xiao, K., & Tedrake, R. (2017). Evaluating robustness of neural networks with mixed integer programming. [arXiv:1711.07356](https://arxiv.org/abs/1711.07356).
- Tramer, F., Carlini, N., Brendel, W., et al. (2020). On adaptive attacks to adversarial example defenses. *Advances in Neural Information Processing Systems*, 33, 1633–1645.
- Varley, J., & DeChant, C., Richardson, A., et al. (2017). Shape completion enabled robotic grasping. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, pp. 2442–2447.
- Wang, F., Zhang, C., & Xu, P., et al. (2022). Deep learning and its adversarial robustness: A brief introduction. In *Handbook on Computer Learning and Intelligence: Volume 2: Deep Learning, Intelligent Control and Evolutionary Computation*. World Scientific, pp. 547–584.
- Wang, S., & Pei, K., Whitehouse, J., et al. (2018). Efficient formal safety analysis of neural networks. In *Proceedings of Neural Information Processing Systems*, pp. 6369–637.
- Weng, L., Zhang, H., & Chen, H., et al. (2018). Towards fast computation of certified robustness for relu networks. In *International Conference on Machine Learning, PMLR*, pp. 5276–5285.
- Wicker, M., & Kwiatkowska, M. (2019). Robustness of 3d deep learning in an adversarial setting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11,767–11,775.
- Wong, E., & Kolter, Z. (2018). Provable defenses against adversarial examples via the convex outer adversarial polytope. In *International Conference on Machine Learning, PMLR*, pp. 5286–5295.
- Wu, Z., Song, S., & Khosla, A., et al. (2015). 3D shapenets: A deep representation for volumetric shapes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1912–1920.
- Xiang, C., Qi, C. R., & Li, B. (2019). Generating 3d adversarial point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9136–9144.
- Xu, P., Ruan, W., & Huang, X. (2022). Quantifying safety risks of deep neural networks. *Complex & Intelligent Systems* 1–18.
- Yang, J., Zhang, Q., & Fang, R., et al. (2019). Adversarial attack and defense on point sets. [arXiv:1902.10899](https://arxiv.org/abs/1902.10899).
- Zhang, H., Weng, T. W., & Chen, P. Y., et al. (2018). Efficient neural network robustness certification with general activation functions. In *Proceedings of Neural Information Processing Systems*, pp. 4944–4953.
- Zhang, Y., Liang, G., & Salem, T., et al. (2019). Defense-pointnet: Protecting pointnet against adversarial attacks. In *2019 IEEE International Conference on Big Data (Big Data)*. IEEE, pp. 5654–5660.
- Zhao, Y., Wu, Y., & Chen, C., et al. (2020). On isometry robustness of deep 3D point cloud models under adversarial attacks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1201–1210.
- Zhou, H., Chen, K., & Zhang, W., et al. (2019). Dup-net: Denoiser and upsampler network for 3d adversarial point clouds defense. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1961–1970.

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

## Authors and Affiliations

Ronghui Mu<sup>1</sup> · Wenjie Ruan<sup>2</sup>  · Leandro S. Marcolino<sup>1</sup> · Qiang Ni<sup>1</sup>

Ronghui Mu  
ronghui.mu@lancaster.ac.uk

Leandro S. Marcolino  
l.marcolino@lancaster.ac.uk

Qiang Ni  
q.ni@lancaster.ac.uk

<sup>1</sup> School of Computing and Communication, Lancaster University, Lancaster, UK

<sup>2</sup> Department of Computer Science, University of Exeter, Exeter, UK